

RESOLUTION NUMERIQUE



TP n°2

*Algorithmes de base :
Schéma explicite d'Euler*

Schéma explicite d'Euler : 1er et 2nd ordre

OBJECTIFS DU TP :

Dans le cadre du travail de l'ingénieur (modélisation, traitement de données, résolution d'équations comportementales, simplification de modèles, etc...), on peut être amenés à utiliser des outils de résolution et/ou simulation numériques.

Au-delà des simulateurs présents sur le marché (Matlab, Scilab, Solidworks, etc..), que l'on abordera en TP, il peut être très utile de coder soit même certaines problématiques classiques. Le langage **Python**, privilégié en CPGE est pour cela parfaitement adapté...

Dans ce cadre, le programme de SII a fixé les compétences suivantes :

- Solution d'un système stationnaire par la méthode de **dichotomie** et **Newton** ;
- **Dérivation** et **intégration numérique** ;
- Résolution d'un système linéaire par la méthode du **pivot de Gauss-Jordan** ;
- Résolution d'équation différentielle par la méthode du **schéma explicite d'Euler** ;
- Résoudre un problème en utilisant une solution d'**intelligence artificielle**.

Dans ce 2nd TP, nous allons voir (ou revoir) la méthode d'approximation de la solution d'une équation différentielle. Nous nous limiterons aux 1^{er} et 2nd ordre soumis à aux entrées indicielle (échelon de Heaviside de hauteur 1) dont nous connaissons les modèles...

Afin de gagner en efficacité dans nos algorithmes, nous utiliserons allègrement le module **Numpy**. De même, nous tracerons le plus souvent une image de notre travail à l'aide du module **Matplotlib**. On rappelle dans cette 1^{ère} section les éléments de base de ces modules...

1. RAPPELS DES ELEMENTS DE BASE DE MATPLOTLIB ET NUMPY

On suppose que l'on a effectué les commandes :

 **import matplotlib.pyplot as plt**

```
plt.clf() #efface le tracé précédent
plt.plot(X,Y,label='approchée',color='b') #affiche la solution approchée et précise la légende en bleu
plt.plot(X,Y2,label='exacte',color='r') #affiche la solution exacte et précise la légende en rouge
plt.legend() #affiche la legende approchée et exacte
plt.title('Reponse indicielle') #affiche le titre
plt.grid() #affiche la grille
plt.xlabel('$t$') #affiche t pour l'axe des abscisses
plt.show() #commande l'affichage
```

 **import numpy as np**

```
np.pi #permet d'obtenir la valeur de pi
A=np.array([1,2,3,4],float) #créait un tableau A à une dimension contenant 1,2,3 et 4 définit comme flotants
A[0] #renvoie ici le premier élément de A, soit 1.0
A[1:3] #renvoie les éléments de A compris entre 1 et 3, soit ici 2.,3.
A[0:4:2] #renvoie les éléments de A compris entre 0 et 4 avec un pas de deux, soit ici 1.,3.
A=np.array([[1,2],[1,2]],[[1,2],[1,2]]) #créait un tableau A à trois dimensions toutes de dimension deux
A=np.eye(3,3) #créait une matrice A diagonale de dimension 3*3
A=np.zeros((2,3)) #créait une matrice A nulle de dimension 2*3
A=np.ones((3,2)) #créait une matrice identité de dimension 3*2
A=np.arange(0,1,0.1) #créait une liste allant de 0 à 1 (exclus) tous les 0.1, soit dix éléments
A=np.linspace(1,4,6) #créait une liste de six éléments également répartis allant de 1 à 4 (inclus)
A=np.random.rand(n,p) #créait une matrice de taille (n,p) à coefficients aléatoires uniformes sur [0,1]
B=np.sin(A) #créait une matrice de dimension de A des sinus de A
np.shape(A) #renvoie la dimension de A, ici (n,p)
np.flatten(A) #renvoie une liste (une dimension) des éléments de A
np.reshape(A,p,n) #refond la matrice A(n,p) en matrice A(p,n) (correspond ici à la transposée)
np.transpose(A,p,n) #transpose la matrice A
f(A) #applique la fonction f à tous les éléments de A
```

2. MÉTHODE DU SCHÉMA EXPLICITE D'EULER

2.1 Rappel sur la dérivation numérique :

La méthode consiste à approximer une fonction par la méthode des accroissements finis. Pour ce faire, plusieurs approches sont possibles :

- On utilise la valeur de la dérivée première arrière en t_k : $\frac{dx(t)}{dt} \simeq \frac{x_k - x_{k-1}}{h}$;
- On utilise la valeur de la dérivée première avant en t_k : $\frac{dx(t)}{dt} \simeq \frac{x_{k+1} - x_k}{h}$;
- On utilise la valeur de la dérivée seconde centrée en t_k : $\frac{d^2x(t)}{dt^2} \simeq \frac{x_{k+1} - 2x_k + x_{k-1}}{h^2}$;

2.2 Schéma explicite d'Euler :

En se limitant aux équations différentielles du 1^{er} ou 2nd ordre, on comprend aisément qu'il est possible d'utiliser la méthode d'approximation d'une dérivée et d'une dérivée seconde pour approximer une telle équation...

Application à l'étude d'un 1^{er} ordre :

On rappelle les études en SII sont en général menées autour d'un point de fonctionnement, donc en considérant les **conditions initiales** comme **nulles**.

En conséquence, et si on prend une équation différentielle du 1^{er} d'une fonction $s(t)$ soumise à une entrée $e(t)$, on a :

$$s(t) + \tau \cdot \frac{ds(t)}{dt} = K \cdot e(t)$$

Qui peut s'écrire par l'approximation de la dérivée 1^{ère} avant :

$$S_{k+1} + \tau \cdot \frac{S_{k+1} - S_k}{h} = K \cdot E_k$$

Ou encore :

$$S_{k+1} = \frac{K \cdot h \cdot E_k + \tau \cdot S_k}{h + \tau}$$

Ceci correspond à une suite récurrente qu'on se propose de programmer...

Application à l'étude d'un 2nd ordre :

La méthode est identique :

$$s(t) + \frac{2\xi}{\omega_0} \cdot \frac{ds(t)}{dt} + \frac{1}{\omega_0^2} \cdot \frac{d^2s(t)}{dt^2} = K \cdot e(t)$$

Qui peut s'écrire par l'approximation de la dérivée 1^{ère} avant et dérivée seconde :

$$S_{k+1} + \frac{2\xi}{\omega_0} \cdot \frac{S_{k+1} - S_k}{h} + \frac{1}{\omega_0^2} \cdot \frac{S_{k+1} - 2S_k + S_{k-1}}{h^2} = K \cdot E_k$$

Ou encore :

$$S_{k+1} = \frac{K \cdot \omega_0^2 \cdot h^2 \cdot E_k + 2 \cdot S_k (\xi \cdot h \cdot \omega_0 + 1) - S_{k-1}}{\omega_0^2 \cdot h^2 + 2 \cdot \xi \cdot h \cdot \omega_0 + 1}$$

3. ALGORITHMES DU SCHEMA EXPLICITE D'EULER

Dans les deux cas : 1^{er} et 2nd ordre, on se place dans le cas où les **conditions initiales sont nulles** et pour une étude à partir d'un temps $t_0=0$.

3.1 Premier ordre :

🖥 Ecrire une fonction **euler_1(f, k, tau, h, a)** qui renvoie la solution approchée **s(a)** à une entrée **e(t)=f(t)** d'une équation différentielle du 1^{er} ordre de gain **k** et de constante de temps **tau** pour une approximation d'Euler de pas **h**.

Indice : Le nombre d'itération **n** pourra s'écrire : **int(a/h)**.

🖥 Tester votre fonction.

On pourra par exemple tester avec la fonction de Heaviside, ici $f=1$ en $3 \times \text{tau}$ pour différents n ...

🖥 Ecrire un script permettant de tracer sous **matplotlib** la solution approchée et connue de la réponse **s(t)** pour une entrée $e(t) = u(t)$ avec des conditions initiales nulles.

Le script devra réaliser les opérations suivantes :

- Importer le **module numpy** sous l'alias **np** ;
- Importer le **module matplotlib.pyplot** sous l'alias **plt** ;
- Initialiser par **input** les variables **k**, **tau**, **n** (nombre de subdivisions), **t=a** (temps d'étude) et la fonction **f**.
- Créer une liste **X** des différents **t_k** ;
- Créer une liste **Y** (de la taille de X) des différentes solutions approchées de **s(t_k)** à partir de la fonction **euler_1** ;
- Créer une liste **Y2** (de la taille de X) des différentes solutions exactes de **s(t_k)** ;
- Tracer les solutions approchées et exactes de **s(t)** ;

🖥 Faire varier le nombre n d'itération et observer la différence entre la courbe réelle et celle approchée...

🖥 Effacer la partie du script qui trace la solution exacte de la réponse à un échelon de Heaviside.

🖥 Tester maintenant votre script avec les autres fonctions classiques : **e(t)=a.t.u(t)** et **e(t)=a.sin(ω.t).u(t)**.

Indice : Pour la fonction $\sin(\omega.t)$, on pourra rajouter un argument optionnel à la fonction **euler_1**, ainsi qu'un **input** d'**omega**...

🔒 **Enregistrez votre script sous le nom : nom_prenom_trace_euler1**

3.2 Second ordre :

Réaliser un travail similaire pour une équation différentielle d'ordre 2 !